

Embedded Project Report

1. Code Architecture

The system is designed as an event-driven finite state machine (FSM) running on ESP32. Instead of using *delay()* that blocks the processor, all timing is controlled using *millis()*, allowing multiple tasks to execute concurrently.

Finite State Machine (FSM) controls the entire behaviour of the access control system.

System States:

- WAIT - Holds the display for sometime and resets
- TYPING - Default state. Enters WAIT everytime a 7s idle is observed
- VALIDATING - Achieved when “#” key is hit. Checks if the given input matches any pre-existing ID.
 - If it is valid it enters ACCESS_GRANTED
 - Otherwise checks if error count >3
 - If yes, enters LOCKOUT
 - Otherwise, increments error count, holds display in the same state and resets
- ACCESS_GRANTED - If a match is found the details are logged and appropriate message is displayed
- LOCKOUT - After 3 incorrect attempts, the processor ignores all inputs for some time then enters WAIT
- EMERGENCY - On hitting the pushbutton the system goes into override. Permanent access is given and the only way to bypass this is to issue a hard reset

Keypad States:

- * - Backspace
- # - Enters VALIDATING
- Otherwise just appends the entered key and displays it through a mask

Display Format:

- Green LED for access granted
- Red for denied entry and lockout

Network System:

- Connects to Wokwi's Guest WiFi and handles errors appropriately
- NTP is responsible for collecting time of logging
- When access is granted, the details are uploaded to a Static JSON document

This modular design makes the software easy to debug and extend.
The code is also integrated with easy-to-interpret functions at each stage

2. Offline Queue Structure

An array based LIFO stack is used.

When the network drops, 10 logs are pushed into the *offlineQueue* array.

When Wi-Fi is restored, the *flushQueue()* function iterates from the last stored index until the zeroth and fires the data to the cloud until the stack is entirely empty, preventing any data loss during blackouts.

3. Memory Management

Instead of using a standard dynamic memory allocation which can leave fragmented holes in the ESP32's limited heap memory over time, we have used a *StaticJsonDocument<200>*.

This allocates exactly 200 bytes on the stack rather than the heap. The moment the *HTTP POST* function completes, that memory block is instantly and cleanly freed, guaranteeing the terminal can run for years without suffering a memory-leak crash.